

Chapter 1

Course Introduction & Embedded Systems Foundations

ENEL712 Embedded Systems Design — Lecture 1

Summary

Foundational overview of embedded systems: AT90USB-style 8-bit AVR architecture, memory hierarchy, communication protocols, real-time/resilience principles, and hardware/software partitioning.

Topics Covered

- MCU architecture fundamentals (RISC, pipelining, Harvard vs Von Neumann)
- Memory types and constraints (Flash, SRAM, EEPROM)
- Communication protocols (UART, SPI, I2C)
- Real-time systems and defensive software
- Hardware/software partitioning

1.1 MCU Architecture Fundamentals

The Atmel AT90USB series uses the 8-bit AVR RISC architecture.

1.1.1 RISC (Reduced Instruction Set Computer)

- Large register file (32 general-purpose registers) and a simplified instruction set.
- Most instructions execute in a single clock cycle, enabling up to 16 MIPS at 16 MHz.
- Load-store architecture: operations occur only on registers, minimising slower SRAM traffic.

1.1.2 Two-Stage Pipelining

- While one instruction is executed, the next is fetched from Flash, allowing effective single-cycle execution.
- Branch instructions or interrupts flush the pipeline, costing 1–4 cycles.

- For time-critical code (e.g., ISRs), branching should be minimised.

1.1.3 Harvard vs. Von Neumann

- AVR uses a Harvard architecture: separate buses for program (Flash) and data (SRAM).
- Allows simultaneous instruction fetch and data access — avoids the Von Neumann bottleneck.
- Reason for using PROGMEM to keep constants in Flash and save SRAM.

1.2 Memory Types and Constraints

Embedded systems must operate within strict memory constraints, requiring careful management of three primary memory types.

Memory	Size	Volatile?	Use	Key constraint
Flash	128 KB	No	Program code & constants	100,000 write cycles; use PROGMEM for tables
SRAM	8 KB	Yes	Variables, stack, buffers	Stack grows down from 0x20FF; risk of overflow
EEPROM	4 KB	No	Settings & calibration	~3.4 ms/byte write; 100,000 cycles/byte

EEPROM is unsuitable for frequent logging or writes inside time-critical ISRs. For data written more than once a day, use external storage or wear leveling.

1.3 Communication Protocols

1.3.1 UART (Asynchronous)

- Simple point-to-point serial using two wires (TX/RX).
- Both sides agree on a baud rate; frame = START (0), 8 data, STOP (1).
- Requires <2% clock error to avoid corruption — timing errors accumulate.

1.3.2 SPI (Synchronous)

- Fast, full-duplex; minimum four wires (SCK, MOSI, MISO, CS).
- Ideal for SD cards/displays; needs a CS line per slave.
- Master and slave must match Clock Polarity (CPOL) and Clock Phase (CPHA) modes.

1.3.3 I²C (Synchronous)

- Two-wire bus (SCL/SDA) supporting up to 127 devices via addressing.
- Open-drain lines with pull-up resistors (e.g., 4.7 k Ω).
- Transaction = START → 7-bit address → R/W bit → ACK from slave.

1.4 Advanced Concepts and Resilience

- Real-Time Systems: deterministic, bounded execution. Hard real-time (e.g., automotive) must meet all deadlines; standard OSs introduce scheduler jitter.
- Defensive Software: checksums for data integrity, watchdog timer (WDT) to recover from runaway code.
- Hardware/Software Partitioning: prefer built-in hardware (Timer PWM, Input Capture, DMA) over software loops to maximise CPU throughput.

1.5 Use Cases & Applications

An 8-bit AVR like the AT90USB shines when you need deterministic timing, low cost, and tiny power budgets — not raw throughput. Typical embedded applications:

- Battery sensor nodes (e.g., the vineyard weather station): MCU sleeps for seconds, wakes via WDT, samples I²C sensors, sends a packet over UART/wireless, then sleeps.
- Real-time motor control: tight ISRs that must hit deadlines every PWM cycle — RISC + 2-stage pipeline gives predictable single-cycle execution.
- USB peripheral devices: AT90USB has a built-in USB peripheral, so it's a good fit for HID gadgets (keyboards, custom controllers).
- Safety-critical glue logic: a small MCU + watchdog timer monitoring a larger system, ready to reset or report on failure.

1.5.1 When NOT to use an 8-bit AVR

- FPU-heavy DSP, audio, or PID with float — pick a Cortex-M4 with FPU.
- More than ~128 KB code or ~8 KB RAM — pick an ARM Cortex-M with more memory.
- Full-speed USB / Ethernet / TCP-IP — pick an ARM Cortex-M or an MPU running Linux.

1.6 Worked Examples

1.6.1 Pipeline flush cost

An ISR services an event every 100 μ s on a 16 MHz AVR. The ISR contains 30 instructions, of which 4 are branches that each cost a 2-cycle pipeline flush.

- Base instructions: 30 cycles.
- Pipeline flush penalty: $4 \times 2 = 8$ extra cycles.
- Total: 38 cycles $\rightarrow 38 / 16e6 \approx 2.375 \mu$ s per ISR — about 2.4 % CPU load.
- Lesson: collapsing branches inside hot ISRs reclaims ~20 % of the routine.

1.6.2 EEPROM endurance

EEPROM cell endurance is 100,000 cycles per byte. Logging a 16-byte record once a minute would burn through that in ~70 days. Either drop to once an hour, write to external SPI flash, or implement wear leveling that rotates the write address across, say, 100 EEPROM positions for ~7,000 days.

1.7 Code Snippets

1.7.1 Storing a constant in Flash with PROGMEM

```
#include <avr/pgmspace.h>

const uint8_t sineLUT[256] PROGMEM = { /* ... */ };

uint8_t sample(uint8_t i) {
    return pgm_read_byte(&sineLUT[i]); // reads from Flash, not SRAM
}
```

1.7.2 Petting the watchdog

```
#include <avr/wdt.h>

int main(void) {
    wdt_enable(WDTO_500MS); // reset MCU if not petted within 500 ms
    for (;;) {
        do_work();
        wdt_reset();        // "pet the dog"
    }
}
```

Key Definitions

MIPS

Millions of Instructions Per Second; a measure of CPU throughput.

Pipeline Flush

Performance cost when a branch/interrupt forces the CPU to discard prefetched instructions (1–4 cycles).

Modified Harvard

Architecture that separates program and data buses but allows some cross-access (e.g., PROGMEM).

Open-Drain

Bus type used in I²C where devices only pull lines low; requires pull-up resistors.

Non-Volatile

Memory that retains data when power is removed (Flash, EEPROM).

Volatile

Memory that loses data when power is removed (SRAM).

PROGMEM

C directive that stores constant data in Flash instead of consuming SRAM.

Von Neumann Bottleneck

Limitation of single shared bus that blocks simultaneous fetch and data access.

Watchdog Timer (WDT)

Hardware timer that resets the MCU if not 'petted' periodically; recovers from runaway.

ISR

Interrupt Service Routine; function executed in response to a hardware signal.

Stack Overflow

Fault when downward-growing stack collides with upward-growing globals.

MIPS Throughput

Approximation of how many simple instructions a CPU finishes per second; AT90USB peaks around 16 MIPS at 16 MHz.

Wear Leveling

Spreading EEPROM/Flash writes across many cells so no single cell hits its endurance limit prematurely.

RTC

Real-Time Clock — low-frequency oscillator (often 32.768 kHz) used to keep wall-clock time and wake from deep sleep.

Sources

- Lecture 1.pdf